

42 트랙 — C/C++ CS 기본 → 취업용 웹 캡스톤(pong)

이 트랙은 무엇인가. 42 성격 그대로, C/C++로 컴퓨터 기본기(프로세스·메모리·시그널·동시성·템플릿·소켓)를 손으로 쌓고, 마지막은 **취업으로 직결되는** **풀스택 웹 프로젝트** **pong-pong** 으로 달는다. 코드 *재현*이 아니라 *판단력* — L3(아래)만 백지에서 다시 만들고 나머지는 빠르게 통과한다.

읽는 법. 위에서 아래로 한 프로젝트씩 헤쳐나가면 그대로 완주된다. 각 프로젝트 블록은 **자족적**이다 — 그 프로젝트의 노트 순서·재구성 방법·L3·검증·함정·완료 바·커밋법이 블록 안에 다 있다. 위로 되돌아갈 필요 없다. (학습 철학·전체 지도는 [../STUDY.md](#), 면접 L3 색인은 [../INTERVIEW.md](#).)

딱 한 번만 외우는 공통 어휘 (이후 각 블록은 이 말만 쓴다): - L3 = 동시성·메모리 소유권·시그널 안전·예외 안전·프로토콜 등 *폭발 변경 큰 결정*. **답지 덮고** 백지에서 코드+이유를 재구성하고, 실패 모드까지 말로 방어돼야 “끝”이다. - L2 = 구조·흐름·책임 분리. 답지 펼쳐 설명되면 충분(재구성은 선택). - L1 = 보일러 플레이트·문법·설정. 읽고 통과. 다시 치지 않는다. - 핵심 동작 = docs/commits/NNN.md (답지)를 덮고, docs/practice/NNN.md (문제지)의 계약만 보고 직접 구현 → 답지와 diff → 네 결정이 다르면 왜 다르지 따진다. 이 “백에 부딪히는 순간”이 학습이다. - 각 답지 끝의 **## 기억/설명 Level** 에 그 커밋의 L3/L2/L1이 적혀 있다. 돌아가기 전에 그것부터 본다.

전체 경로:

linux-admin(M0) → small-shell → format-printer → signal-message-bus → thread-dining
→ stack-sort → stl-container → irc-relay-server → ★pong-pong(캡스톤)

왜 이 구성인가(범위 결정). CS 기본에 필요한 것만 남겼다. ray-scene-tracer 제외(그래픽스 적용 연습이라 새 CS 개념이 적음), game-server-foundations/reliability 제외(시스템서버 심화는 기본 범위 밖). 서버는 irc 하나만(BSD 소켓·네트워크 프로토콜 = CS 핵심). pong이 끝나면 **백엔드** (backend.md) 또는 **프론트**(frontend.md)로 분기한다. pong에서 TypeScript를 익히므로 프론트의 TS 선행이 자연히 채워진다.

1. linux-admin — 작업 규율 게이트(M0)

무엇/왜. 모든 경로의 #0. Git-Conventional Commits-Markdown 문서 규율 + Born2beroot's Linux 운영 정책(사용자·SSH·sudo·방화벽)을 *선언하고 검증* 하는 레포다. 여기서 익힌 커밋·문서 규율이 이후 8개 레포 전부에서 그대로 쓰인다 — 그래서 첫 번째다. 이 트랙에서 얻는 것. “어떻게 커밋하고 어떻게 문서를 남기는가”의 몸매 벤치마크.

장르·리듬. 장르 ③ **작업규율 제화형**(재구성 아님) · 순차(2B). 답지 13면 중 문제지는 2면뿐 — 비유 코드가 거의 없다. **덮어 채우는 게 아니라, 절차를 읽고 네 환경에서 실제로 한 번 한다**.

노트 읽는 순서 ([linux-admin/notes/](#)). [git-cli](#) → [conventional-commits](#) → [github-workflow](#) → [markdown](#). 네 편을 읽고 **바로 적용** — 이 레포를 도는 동안 *네 커밋 자체*가 규율 연습이다.

재구성·커밋 활용법. 답지를 소스처럼 읽지 말고, 체크포인트 적출 순서(시스템 구성도 → 사용자 → SSH → sudo → 방화벽)를 따라가며 *정책 문서를 네 손으로 다시 쓴다*. 004(hardening 체크리스트·검증기)에서 **SSH 정책 = 접속 통제 / sudo = 권한 상승**을 분리해 설명할 수 있어야 한다.

L3 — 손으로 백지 재구성할 것. 가법다. 코드가 아니라 *규율*. “SSH 제한·sudo 로깅·방화벽 허용 포트를 왜 이렇게 두나”를 한 번 실제로 적용해 보는 것.

검증. [make test](#) (문서 키워드 검증) + [scripts/check-commit-msg.sh](#) · [check-timeline.sh](#) · [validate_admin_lab.py](#). (검증기는 문서 grep 수준이라 실제 SSH/sudo 동작까진 안 봄.)

함정. “답지 덮고 코드 채우기”를 기대하지 마라(문제지 2면뿐, 장르가 다르다). 실제 VM 구성이 아니라 *정책 선언*이 핵심.

완료 바 → **다음으로**. 다음 커밋부터 **규율이 저절로 지켜지**면 제화된 것. 영어 명령형 제목 + [근거]/[변경]/[검증] 본문이 손에 붙었으면 [small-shell](#) 로.

커밋 메모. 여기가 커밋 규율의 원천이다. 형식: <type>(<scope>): <영어 명명형> + 한국어 [근거]/[변경]/[검증] 본문(trivial 면제), **AI 트레일러 없음**, KST 09:00~21:59, 분·초 랜덤·중복 금지. 이 규칙은 이후 전 레포 공통이라 각 블록에서 반복하지 않는다 — 정본은 [../commit-policy.md](#).

2. small-shell — C 대장(미니 셸)

무엇/왜. C99 단일 바이너리 미니 셸. 파싱→확장→실행→builtin 상태를 한 프로그램에서 다룬다. C 트랙의 대장(가장 깊은 선행) — 여기서 프로세스·fd·파이프·fork/exec를 손에 익히면 결과지들이 풀린다. 이 트랙에서 얻는 것. “누가 fd를 열고 누가 닫나”를 소리 내 말할 수 있는 메모리·프로세스 소유권 감각.

장르·리듬. 장르 ①(코드 재구성, 일부 따라만들기) · 순차(2B). 답지 11 = 문제지 11(1:1) — **덮고 본문을 채우는** 전형적 재구성 레포. [docs/notes/](#) **없음** → 막히면 루트 notes/ 1부로 대체.

노트 읽는 순서 ([small-shell/notes/](#)). c-language (처음 보는 스택이면 1부 개념부터 깊게) → [gnu-readline](#). 짝 [reference-impl/](#) 를 한 번 빌드해 손에 익힌다.

재구성·커밋 활용법. 커밋을 세 묶음으로 본다 — ① 기반(002~003: [shell.h](#) 공용 자료구조, 환경 저장소), ② 입력 정제(004~006: lexer→parser tree→expansion), ③ 실행(007~009: stdin 루프→builtin→fork(pipe/exec). **004~006과 007~009가 L3다**. 문제지를 덮고 lexer/parser/executor 계약을 직접 세운 뒤 답지와 diff.

L3 — 손으로 백지 재구성할 것. fork-exec와 **pipe fd 관리**(누가 닫나), quote/expansion 섞인 lexer, **builtin은 부모 프로세스에서 실행**(상태 변경 경계), 파이프라인 종료 상태 보존, heredoc/redirection 결합.

검증. [make](#) → [make test](#) (smoke.sh가 builtin-quote-expansion-pipeline-redirection-heredoc-syntax-error 시나리오 검증).

함정. job control-wildcard-command substitution-subshell-arithmetic은 **범위 밖**. 범위를 “한 줄 명령 실행 + builtin 상태”로 고정하고 육신내지 마라.

완료 바 → **다음으로**. 빈 파일에서 fork/pipe/exec 경로와 builtin 부모 실행 경계를 **백지로 다시 짜고** fd 소유권을 말로 방어되면 다음. 그 다음 4개(결가지)는 small-shell에서 익힌 C를 *각 주제*로 *쭈뼌* 적용한다.

커밋 메모. dual-form 스크립. 답지 docs(commits) / 문제지 docs(practice) 분리 커밋 변경 파일만 [add\(git -C small-shell add docs/commits docs/practice\)](#), 전체 [-A](#) 금지.

3. format-printer — C 결가지 ①: 가변인자·포매팅(ft_printf)

무엇/왜. 42 [ft_printf](#). 포맷 문자열을 파싱해 변환 지정자·플래그를 처리하고 정적 라이브러리 [libftprintf.a](#) 로 제출. [va_list](#) 와 플래그 우선순위는 **좁고 깊은** 주제 하나를 판다. 이 트랙에서 얻는 것. 가변인자 안전 전달 + width/precision/flag의 우선순위 추론.

장르·리듬. 장르 ④ 따라만들기 경향(scaffold-first) + ① · 순차(2B). 답지 14 / 문제지 13.

노트 읽는 순서 ([format-printer/notes/](#)). ar (정적 라이브러리 아카이브) → [c-test-harness](#). 각 노트에 짝 reference-impl이 있다.

재구성·커밋 활용법. 000(공개 API·빌드 규칙 골격, L1) → 001(counted-write 컨텍스트 [t_printf](#), L2: 모든 변환이 같은 오류 반환) → **002(포맷 필드 파서, L3) → 003~005(변환 9종 dispatch) → 006~008(width 정렬·precision/0-padding·부호/alternate, L3: precision-vs-zero-flag 우선순위) → 009~011(libc 비교·오류 처리)**. 002와 006~008을 덮고 직접 켜다.

L3 — 손으로 백지 재구성할 것. % 뒤 **flags→width→precision→specifier** 순서 소비, 변환 9종과 flag 조합의 우선순위(width→precision 상호작용, 부호+0패딩 동시), [va_list](#) 전달, counted-write sticky error.

검증. [make](#) → [make test](#) (sprintf와 바이트·반환값 비교).

함정. length modifier-locale-stdout 외 출력 첫 커밋 체크리스트 밖 변환은 범위 밖.

완료 바 → **다음으로**. width/precision/flag 우선순위를 백지에서 표로 재현하고 sticky error를 설명되면 다음.

4. signal-message-bus — C 결가지 ②: 시그널·IPC(minitalk)

무엇/왜. 42 [minitalk](#). SIGUSR1 / SIGUSR2 두 시그널만으로 임의 문자열을 비트 스트림으로 신뢰성 있게 전달하고, ACK/NACK-timeout으로 강화. 시그널 안전과 race-free 대기를 판다. 이 트랙에서 얻는 것. signal-safe 코드와 비동기 핸들러 경계 감각.

장르·리듬. 장르 ④ 따라만들기(경계 선택기 프로토콜 어휘) · 순차(2B). 답지 12 / 문제지 11.

노트 읽는 순서 ([signal-message-bus/notes/](#)). posix-signal (정독) + 짝 reference-impl.

재구성·커밋 활용법. 000(프로토콜 어휘 헤더: [MT_ZERO_SIGNAL / MT_ONE_SIGNAL / MT_ACK_SIGNAL](#), L2) → 001(server PID 출력·[pause\(\)](#)) → **002~003(MSB·FIRST 바이트 조립·byte→bit 송신, L3) → 004(NUL 종료자=메시지 경계, L3) → 005~006(sigsuspend 기반 ACK=ACK timeout, L3) → 007(경쟁 sender 거절=NACK, L3: 단일 활성 송신자) → 008~010(고정)**. 002~007이 L3 핵심.

L3 — 손으로 백지 재구성할 것. signal-safe 비트 조립, **MSB-first 순서 맞물림**, [sigprocmask / sigsuspend](#) 로 **빠른 ACK 누락 방지**(race-free 대기), ACK timeout으로 무한 대기 차단, 경쟁 sender 소유권.

검증. [make test](#) (실제 server를 백그라운드 기동, client 전송 결과를 stdout에서 비교).

함정. 전송 속도 최적화가 아니라 **ACK 기반 안정성**이 목표. 외부 라이브러리가 추가 디렉터리 금지.

완료 바 → **다음으로**. [sigsuspend](#) 대기 루프와 ACK timeout 상태 모델을 백지로 짜고 “왜 race가 안 나나”를 방어되면 다음.

5. thread-dining — C 결가지 ③: pthread·동시성(philosophers)

무엇/왜. dining philosophers, pthread-mutex-monitor 패턴으로 교착(deadlock)을 피하며 동시성을 푼다. 이 트랙에서 얻는 것. 락 소유권·교착 회피·상태 vs 출력 분리 — 동시성 L3의 첫 본격 훈련.

장르·리듬. 장르 ④ 따라만들기(골격 선택기 공통 타입) · 순차(2B). 답지 19 / 문제지 17.

노트 읽는 순서 ([thread-dining/notes/](#)). posix-pthread → posix-time-io → awk (로그 검증).

재구성·커밋 활용법. 000(공통 타입 [t_config / t_philo / t_table](#), L2: [table](#)이 fork/philo 소유) → 001~002(ms helper-deadline 플링) → **003~004(state_mutex vs print_mutex 책임 분리, L3) → 005~008(철학자 루틴 출력 fork 순서·monitor·수명주기, L3) → 009~012(단일 철학자·meal limit 경계) → 013~017(고정)**. 003~008이 L3 — 덮고 직접 구현 후 답지와 diff.

L3 — 손으로 백지 재구성할 것. **출력 fork 횟동 순서로 deadlock 회피**, [state_mutex](#) (종료 플래그·식사 상태) vs [print_mutex](#) (로그) 분리, **monitor는 main 스레드에서 직접 실행**, deadline 기반 sleep 루프와 meal limit 즉시 종료.

검증. [make test](#) (실제 바이너리 실행, 종료 코드·로그 정규식 검증).

함정. bonus(semaphore/process)는 *실패*하는 target. 철학자 200명 초과는 거절. 로그 순서 결정성은 보장 안 됨(타이밍 의존).

완료 바 → **다음으로**. 교착이 왜 안 나는지(출력 순서)와 두 mutex의 책임 경계를 백지로 방어되면 다음.

6. stack-sort — C 결가지 ④: 정렬 알고리즘(push_swap·Python)

무엇/왜. 42 [push_swap](#). 11개 스택 명령만으로 정렬하는 C 바이너리 + 검증 C 바이너리 + Python 테스트 러너. 알고리즘 설계와 “독립 적용기로 출력 검증하는 법”을 함께 배운다. 이 트랙에서 얻는 것. 좌표 압축·radix 같은 알고리즘 사고 + 검증 자동화. (이 트랙의 유일한 Python 접점.)

장르·리듬. 장르 ③ 순계산(알고리즘 설계) + ④ 따라만들기 · 순차(2B). 답지 23 / 문제지 21.

노트 읽는 순서 ([stack-sort/notes/](#)). python (C 프로그램을 구동·검증하는 도구 측면).

재구성·커밋 활용법. 000~001(이중 바이너리 공유 소스 계약, L3: 공통 stack 연산 공유) → 002~004(스택 저장소·swap/push-rotation, L3: values와 ranks 나란히 보관) → **005~007(파서: 토른·signed int 검증·중복 거절·rank 압축, L3: 좌표 압축) → 008~012(tiny sort·LSD radix-main orchestration, L3: 이원화·emit 플래그) → 013~015(checker) → 016~020(자동 검증)**. 알고리즘 핵심(005~012)을 손으로 다시 설계한다.

L3 — 손으로 백지 재구성할 것. **좌표 압축**(값 범위 무관 radix), LSD radix 전략, tiny sort 패턴 분기, 두 바이너리가 같은 primitive 공유, emit 플래그로 출력 제어.

검증. [make](#) → [make test](#) (Python 러너: 파서/checker/정렬/stress).

함정. 명령 수 *최적화*가 목표가 아니라 *안정적 정렬 전략*. int 범위만. 시간 압박이면 결가지 중 **이 한 편만 남기고 축소**해도 된다(STUDY “떼어쓰기”).

완료 바 → **다음으로**. 좌표 압축 + radix를 백지로 재설계하고 checker와의 분리를 설명되면, 이제 C++로.

7. stl-container — C++ 대장(C++98 STL 재구현)

무엇/왜. C++98 header-only [vector](#) · [stack](#) · [map](#) 을 표준 라이브러리 작성자의 도구만으로 재구현. C++11 이후 편의 없이 만든다. C++ 트랙의 대장. 이 트랙에서 얻는 것. 템플릿·allocator-iterator-예외 안전 — C++의 진짜 어려운 부분.

장르·리듬. 장르 ①(말바닥 재설계) · 순차(2B). 답지 29 / 문제지 25 — 본격 코드 재구성 레포.

노트 읽는 순서 ([stl-container/notes/](#)). cpp98 (C++98 도구 심화) + 짝 reference-impl.

재구성·커밋 활용법. 000~001(빌드 골격 —std=c++98 고정, L2) → **002~005(type traits·pair·algorithm·iterator, L3: [enable_if / is_integral](#) SFINAE, [reverse_iterator](#)) → 006~009(vector 저장소·stack adapter·map red-black tree, L3: allocator rebind·RB 균형) → 010(std 컨테이너 비교) → 011~022(overflow-aliasing 방어·iterator 호환·회귀)**. 002~009가 심장.

L3 — 손으로 백지 재구성할 것. allocator 패턴과 원소 생명주기, SFINAE **[enable_if](#) 분기**, [iterator_traits](#) 와 raw pointer specialization, **red-black tree 균형 보정**, self-aliasing range assign/insert, const/mutable iterator 호환, 예외 안전(construct 예외·강한 보정).

검증. [make](#) (테스트 바이너리 빌드 → std 컨테이너와 행동 대조).

함정. header-only(설치형 라이브러리 산출물 아님), C++11 미사용, vector/stack/map만, 성능/벤치마크 밖. 이제 **★심화** 봉우리 — 분량보다 **L3를 깊게**.

완료 바 → **다음으로**. allocator 생명주기·SFINAE·RB 균형을 백지로 재구성하고 self-aliasing을 방어되면, C++17 + 소켓으로.

8. irc-relay-server — C++17: BSD 소켓·네트워크 프로토콜(ft_irc)

무엇/왜. C++17 단일 프로세스 IRC 서버. event loop([kqueue / epoll](#))·partial frame buffering·heartbeat/rate limit guard를 직접 켜다. 이 트랙의 유일한 서버이자 CS의 네트워크 축. 이 트랙에서 얻는 것. 이벤트 루프·backpressure:프로토콜 경계 — pong의 WebSocket 서버로 바로 이어진다.

장르·리듬. 장르 ①(프로토콜을 이벤트 루프 위에 구현) + ④ · 순차(2B). 답지 31 / 문제지 25.

노트 읽는 순서 ([irc-relay-server/notes/](#)). cpp17 → [posix-bsd-sockets](#) → [gnu-make](#) → [e2e-smoke-harness](#). (st에서 C++98을 했으니 cpp17 은 2부 중심으로 빠르게.)

재구성·커밋 활용법. 000~001(범위 기준점) → **002~006(EventManager 인터페이스·kqueue/epoll·Connection·Server, L3: event backend 추상화·fd 소유권·read buffer-outbound queue 관리·write 한곳) → 007~010(IRC parser-reply formatter·Channel-dispatch, L3: protocol/application 경계·backpressure) → 011~013(TCP 서버) → 014~016(action API·chat-metrics-tournament·admin-presence, L3: [AppRepository](#) 계약·[auth](#) 경계) → 064~084(truth fix·세션·토른 안정화), L3는 “서버 권위”와 “shared 계약·[auth](#) 경계” — 이 커밋들만 손으로 재구성하고, CRUD 화면 배신류는 답지 펼쳐 읽고 통과.**

L3 — 손으로 백지 재구성할 것. 서버 권위 경기 상태(점수·위차·승패를 서버가 계산, 클라 입력 불신), GameHub 의 tick loop + 이벤트 기반 broadcast, shared 계약이 포트트/백을 함께 묶는 구조, [AppRepository](#) 로 저장소 추상화 + 트랜잭션, **[auth](#) 경계**(역경 세션 즉시 null, 401 토큰 즉시 제거).

검증. [pnpm -r typecheck/build/test + make test](#) (smoke + Playwright E2E) + Docker Compose 실행. 화면→코드를 왕복하며 실제로 띄워 본다 (pnpm dev).

함정. 외부 OAuth는 환경변수 분리만(저장소 미포함, dev login 기본). 외부 사용자 트래픽용 아님. web unit test는 미완성. **85커밋 전부 재구성은 과투자** — 서버 권위·계약·auth만 깊게.

완료 바 → **트랙 완주**. Docker Compose로 풀스택이 뜨고, 서버 권위 모델·shared 계약·[auth](#) 경계를 백지로 방어되면 42 트랙 완주. GitHub에 push해 커리어 산출물로 남긴다.

커밋 메모. pnpm 모노레포라 변경 범위가 넓다 — dual-form은 docs/commits · docs/practice 만 골라 add, 구현 커밋과 메타·문서 커밋의 **phase** 경계를 섞지 않는다([../commit-policy.md](#) \$날짜·배치).

다음 — 분기, 그리고 병렬성

pong을 닫으면 백엔드([backend.md](#)) 또는 프론트([frontend.md](#))로 간다. 둘 다 M0(linux-admin)- TS(pong)를 이미 끝낸 상태에서 시작하므로, 각 문서는 그 선행을 다시 다루지 않는다.

- **트랙 간 병렬**: 42 스파인 / 백 / 프론트는 서로 **하드 의존이 없다**(공통 게이트는 M0뿐). 그래서 CS(42)와 웹(백·프론트)은 *비대칭*이라 원하면 병렬도 가능하다. 다만 이 트랙 계획은 **42를 먼저 완주한 뒤** 분기하는 단선이다.

- **분기 후 트랙 내부의 병렬성**은 각 문서 말미에서 다룬다(백: 신뢰성→캡스톤 순차 플스, 프론트: 캡스톤이 자족이라 신뢰성과 병행 가능).